

ONEDRAFT

CAD-AI — Aria Powered

Disaster Testing, Security Testing & Operational Readiness Architecture

Version 1.0

Document: OneDraft Disaster Testing, Security Testing & Operational Readiness Architecture Specification

System: OneDraft CAD-AI

Intelligence Layer: Aria

Document Class: Architectural & Implementation Stack Specification

Status: Architectural Specification

Predecessor: Performance, Scalability & Capacity Architecture Specification

Successor: Developer Platform, SDK & Extension Architecture Specification

1. PURPOSE

The Disaster Testing, Security Testing & Operational Readiness Architecture establishes the formal testing and readiness framework required to determine whether OneDraft can withstand failures, security events, infrastructure disruptions, workload anomalies and operational incidents while preserving system integrity.

The purpose of this subsystem is to verify that the preceding OneDraft architectures operate correctly under normal, degraded, adversarial and failure conditions.

Operational readiness shall be demonstrated through repeatable testing rather than assumed from architectural design alone.

2. ARCHITECTURAL ROLE

The Disaster Testing, Security Testing & Operational Readiness subsystem provides the verification boundary between architectural implementation and production operation.

It evaluates:

Failure Recovery

Security Controls

Tenant Isolation

Data Integrity

Service Availability

Backup Restoration

Queue Recovery

Infrastructure Recovery

Operational Procedures

Production Readiness

Incident Response

3. TESTING MODEL

OneDraft shall use layered verification.

UNIT TESTING



INTEGRATION TESTING



CONTRACT TESTING



SYSTEM TESTING



SECURITY TESTING



PERFORMANCE TESTING



FAILURE TESTING



DISASTER RECOVERY TESTING



OPERATIONAL READINESS

Each layer shall verify a distinct class of system behavior.

4. TEST ENVIRONMENTS

Testing shall be separated into controlled environments.

Development

Test

Integration

Staging

Production

Production testing shall use controlled procedures and shall not introduce uncontrolled destructive operations.

5. TEST DATA

Testing shall use deterministic and representative datasets.

Test datasets may include:

Small Projects

Large Projects

Complex Geometry

Multi-Level Buildings

Large Drawing Packages

CAD Imports

BIM Imports

Compliance Rules

Validation Results

Redline Histories

Test data shall not expose customer information unnecessarily.

6. TEST FIXTURES

Reusable test fixtures shall establish known system states.

Fixtures may include:

Organizations

Users

Projects

Models

Drawings

Artifacts

Jobs

Events

Subscriptions

Entitlements

Authorization Policies

Fixtures shall be version-controlled where practical.

7. DETERMINISTIC TESTING

Where deterministic behavior is required, the test environment shall control:

Input Data

Configuration

Model Versions

Code Versions

Rule Versions

Random Seeds

Provider Selection

Time-Dependent Behavior

Deterministic tests shall produce repeatable outcomes.

8. DISASTER TESTING

Disaster testing shall evaluate the system's ability to recover from significant failures.

Scenarios shall include:

Database Failure

Database Corruption

Object Storage Failure

Queue Failure

Worker Failure

API Failure

Network Failure

Service Dependency Failure

Infrastructure Loss

Configuration Failure

Credential Failure

9. DATABASE FAILURE TESTING

Database failure tests shall verify:

Application Detection

Connection Recovery

Transaction Safety

Retry Behavior

Failover

Data Integrity

Recovery Procedures

The system shall not silently continue with inconsistent transactional state.

10. DATABASE RESTORE TESTING

Backup restoration shall be tested periodically.

A restore test shall verify:

BACKUP
↓
RESTORE
↓
DATABASE VALIDATION
↓
SCHEMA VALIDATION
↓
DATA INTEGRITY
↓
APPLICATION STARTUP
↓
FUNCTIONAL VERIFICATION

A backup shall not be considered operationally valid merely because a backup file exists.

11. OBJECT STORAGE FAILURE TESTING

Object storage failure scenarios shall verify:

Artifact Availability

Upload Recovery

Download Recovery

Reference Integrity

Signed Access

Backup Restoration

Artifact Hash Verification

Database metadata shall remain consistent with restored artifacts.

12. QUEUE FAILURE TESTING

Queue failure testing shall verify:

Job Persistence

Message Recovery

Duplicate Handling

Retry Behavior

Dead-Letter Handling

Worker Reconnection

A queue outage shall not silently destroy pending jobs.

13. WORKER FAILURE TESTING

Worker failure shall verify that interrupted jobs are correctly classified and recoverable.

The system shall distinguish:

Pending

Running

Retryable

Failed

Cancelled

Completed

Worker termination shall not create an invalid project state.

14. API FAILURE TESTING

API failure testing shall evaluate:

Instance Failure

Gateway Failure

Service Failure

Timeouts

Connection Resets

Partial Responses

The gateway shall continue routing traffic to healthy service instances where infrastructure permits.

15. NETWORK FAILURE TESTING

Network testing shall evaluate:

Latency

Packet Loss

Connection Failure

Service Partitioning

External Provider Failure

Internal Service Isolation

The system shall fail predictably when network dependencies become unavailable.

16. DEPENDENCY FAILURE TESTING

External and internal dependencies shall be tested independently.

Dependencies may include:

AI Providers

Payment Providers

Object Storage

Email Services

Authentication Providers

External CAD/BIM Services

The failure of an external dependency shall not unnecessarily compromise unrelated OneDraft functionality.

17. AI FAILURE TESTING

Aria and model-provider failures shall include:

Provider Unavailability

Timeout

Malformed Output

Invalid Command

Low Confidence

Provider Switching

Context Construction Failure

Model Response Failure

AI failure shall not directly mutate the Project Model without deterministic command validation.

18. DATA CORRUPTION TESTING

Controlled corruption tests shall verify detection and containment.

Tests may target:

Database Records

Model Artifacts

Drawing Artifacts

Metadata

Event Records

Hashes

Integrity mechanisms shall detect invalid or inconsistent data where designed to do so.

19. TENANT ISOLATION TESTING

Tenant isolation shall be continuously tested.

Tests shall attempt to access another organization's:

Projects

Models

Drawings

Artifacts

Jobs

Billing Records

AI Context

Search Results

Events

Telemetry

Unauthorized access shall be rejected.

20. AUTHORIZATION TESTING

Authorization tests shall cover:

Role Boundaries

Project Membership

Organization Membership

Administrative Access

Privileged Operations

Resource Ownership

Expired Permissions

Removed Membership

Authorization shall be tested independently from frontend visibility.

21. API SECURITY TESTING

API security testing shall evaluate:

Authentication Bypass

Authorization Bypass

Malformed Requests

Parameter Manipulation

Request Smuggling

Replay

Credential Abuse

Rate-Limit Bypass

Payload Abuse

Improper Error Disclosure

22. INPUT SECURITY TESTING

Input validation shall be tested against malformed and unexpected input.

Testing shall include:

Oversized Payloads

Invalid Types

Unexpected Fields

Encoding Errors

Boundary Values

Malformed Files

Invalid Identifiers

Unsupported Formats

Input failures shall terminate safely.

23. INJECTION TESTING

Security testing shall evaluate applicable injection classes.

Examples include:

SQL Injection

Command Injection

Template Injection

Path Traversal

Script Injection

Header Injection

Query Manipulation

The exact applicable test set shall reflect the technologies used by OneDraft.

24. FILE SECURITY TESTING

Imported files shall be tested for:

Malformed Structure

Unexpected Content

Oversized Files

Path Traversal

Unsupported Formats

Corrupt Files

Embedded Threats

File processing shall occur within controlled boundaries.

25. ARTIFACT SECURITY TESTING

Artifact access shall be tested against:

Unauthorized Download

Expired Access

Cross-Tenant Access

Invalid References

Modified Artifacts

Improper Content-Type Handling

Artifact integrity and authorization shall be independently verified.

26. SECRETS SECURITY TESTING

The platform shall verify that secrets are not exposed through:

Logs

Errors

Source Control

API Responses

Telemetry

Build Artifacts

Container Images

Secret rotation procedures shall also be tested.

27. IDENTITY SECURITY TESTING

Identity testing shall cover:

Authentication

Session Expiration

Credential Rotation

Account Lockout

Token Validation

Privilege Changes

Membership Removal

Security-sensitive identity changes shall take effect within defined propagation limits.

28. SECURITY MONITORING TESTING

Security telemetry shall be tested to ensure relevant events are captured.

Events may include:

Authentication Failures

Authorization Failures

Privilege Changes

Administrative Access

Rate-Limit Violations

Suspicious Requests

Credential Changes

Security Configuration Changes

29. PENETRATION TESTING

Periodic penetration testing shall evaluate the externally exposed OneDraft attack surface.

Testing shall include applicable:

Web Applications

APIs

Authentication

Authorization

Infrastructure

Storage

Network Services

Findings shall be classified by severity and tracked through remediation.

30. DEPENDENCY SECURITY TESTING

Software dependencies shall be assessed for known vulnerabilities.

Testing shall cover:

Python Packages

TypeScript/JavaScript Packages

Container Images

Operating System Packages

Database Components

Infrastructure Components

Dependency vulnerabilities shall be evaluated according to exploitability and system exposure.

31. CONTAINER SECURITY

Container images shall be tested for:

Known Vulnerabilities

Unnecessary Packages

Excessive Privileges

Unsafe Configuration

Secret Exposure

Untrusted Base Images

Production containers shall use controlled and versioned images.

32. INFRASTRUCTURE SECURITY

Infrastructure testing shall verify:

Network Segmentation

Firewall Rules

Service Exposure

Credential Controls

Storage Permissions

Database Access

Administrative Interfaces

Only required services shall be exposed.

33. DISASTER RECOVERY OBJECTIVES

The disaster recovery architecture shall validate:

RPO — Recovery Point Objective

RTO — Recovery Time Objective

Each critical subsystem shall have defined recovery objectives.

Recovery objectives shall be measurable through actual restoration testing.

34. RECOVERY TESTING

Recovery testing shall verify:

Detection

Isolation

Restoration

Reconnection

Data Integrity

Service Startup

Functional Verification

User Access

A recovered environment shall not be considered operational until functional verification succeeds.

35. FAILOVER TESTING

Where redundant infrastructure exists, failover shall be tested.

Failover tests may include:

Database Failover

Gateway Failover

Worker Failover

Service Instance Failover

Storage Failover

Failover shall preserve data integrity and authorization boundaries.

36. BACKUP VERIFICATION

Backup systems shall verify:

Backup Completion

Backup Integrity

Backup Encryption

Retention

Restore Capability

Historical Availability

Backup monitoring shall detect failed or incomplete backup operations.

37. CHAOS TESTING

Controlled chaos testing may introduce limited failures into non-production environments.

Scenarios may include:

Service Termination

Worker Termination

Network Latency

Queue Interruption

Database Connection Loss

Dependency Failure

Chaos tests shall have defined scope, expected outcomes and termination conditions.

38. INCIDENT SIMULATION

Operational teams shall periodically simulate incidents.

Exercises may include:

Database Outage

Security Breach

Credential Compromise

AI Provider Outage

Storage Failure

Queue Failure

Tenant Isolation Event

Incident simulations shall test both technical recovery and operational coordination.

39. INCIDENT RESPONSE

OneDraft shall maintain documented incident-response procedures.

The response lifecycle shall include:

DETECT
↓
CLASSIFY
↓
CONTAIN
↓
INVESTIGATE
↓
RECOVER
↓
VERIFY
↓
DOCUMENT
↓
IMPROVE

Incident response shall preserve relevant evidence and audit records.

40. OPERATIONAL RUNBOOKS

Critical services shall have operational runbooks.

Runbooks shall address:

Startup

Shutdown

Failure Detection

Recovery

Rollback

Database Restore

Queue Recovery

Credential Rotation

Service Isolation

Escalation

Runbooks shall be version-controlled.

41. PRODUCTION READINESS

A subsystem shall not be considered production-ready until required readiness criteria are satisfied.

Criteria shall include:

Automated Tests

Security Tests

Performance Tests

Backup Verification

Recovery Testing

Observability

Alerting

Documentation

Rollback Procedures

Operational Ownership

42. RELEASE READINESS GATES

Production releases shall pass defined gates.

BUILD

↓

UNIT TESTS

↓

INTEGRATION TESTS

↓

CONTRACT TESTS

↓

SECURITY TESTS

↓

PERFORMANCE TESTS

↓

MIGRATION TESTS

↓

STAGING VALIDATION

↓

OPERATIONAL APPROVAL

↓
RELEASE

A failed mandatory gate shall prevent release unless explicitly overridden through controlled administrative procedures.

43. ROLLBACK TESTING

Release rollback procedures shall be tested.

Testing shall include:

Application Rollback

Database Migration Recovery

Configuration Rollback

Container Rollback

Artifact Compatibility

Rollback shall not create incompatible Project Model or database states.

44. MIGRATION TESTING

Database migrations shall be tested against representative production-scale datasets.

Migration testing shall verify:

Forward Migration

Rollback Capability, where supported.

Data Integrity

Performance

Lock Duration

Application Compatibility

Migrations shall be treated as production-critical operations.

45. OPERATIONAL ALERTING

Critical conditions shall produce actionable alerts.

Alerts may include:

Service Unavailability

Database Failure

Queue Backlog

Worker Failure

High Error Rate

Security Event

Capacity Exhaustion

Backup Failure

Recovery Failure

Alerts shall identify the affected subsystem and appropriate response procedure.

46. OPERATIONAL OWNERSHIP

Every production subsystem shall have defined operational ownership.

Ownership shall identify:

Responsible Team

Escalation Path

Service Owner

Recovery Procedure

Critical Dependencies

Operational ownership shall remain current as the platform evolves.

47. SECURITY & DISASTER TEST EVIDENCE

Testing shall produce verifiable evidence.

Evidence may include:

Test Results

Logs

Reports

Screenshots

Metrics

Recovery Records

Security Findings

Remediation Records

Approval Records

Test evidence shall be associated with the applicable software, infrastructure and configuration versions.

48. OPERATIONAL READINESS INVARIANTS

The following invariants shall govern the subsystem.

Recoverability: Critical services shall have tested recovery procedures.

Integrity: Recovery shall preserve authoritative data integrity.

Isolation: Failure testing shall preserve tenant boundaries.

Security: Security controls shall be tested rather than assumed.

Traceability: Significant operational actions shall be observable and auditable.

Repeatability: Critical tests shall be repeatable.

Evidence: Readiness decisions shall be supported by verifiable evidence.

Rollback: Production changes shall have tested rollback or recovery procedures.

No Silent Failure: Critical failures shall generate detectable operational signals.

Controlled Testing: Destructive testing shall occur within explicitly controlled boundaries.

49. IMPLEMENTATION REQUIREMENTS

The implementation shall provide:

Automated Test Suites

Security Test Suites

Performance Test Framework

Disaster Recovery Tests

Backup Restore Tests

Failure Injection Framework

Penetration Testing Procedures

Incident Simulations

Operational Runbooks

Production Readiness Checklist

Release Gates

Security Findings Tracking

Recovery Evidence

Operational Metrics

These capabilities shall integrate with the Testing, CI/CD, Infrastructure, Backup, Observability, Identity, Security and Runtime architectures established by preceding stacks.

50. COMPLETION STATUS

The Disaster Testing, Security Testing & Operational Readiness Architecture establishes the formal verification and production-readiness framework for OneDraft.

The subsystem defines disaster testing, database and storage recovery, queue and worker recovery, AI failure testing, tenant isolation testing, authorization testing, API security, injection testing, file security, secrets protection, identity security, penetration testing, dependency security, container and infrastructure testing, RPO/RTO verification, failover, backup restoration, chaos testing, incident simulation, incident response, operational runbooks, production readiness, release gates, rollback testing, migration testing, alerting, operational ownership and evidence preservation.

The architecture establishes the verification layer required to demonstrate that OneDraft is not merely architecturally complete, but capable of being tested, recovered, secured and operated under real production conditions.

The next architectural layer establishes the Developer Platform, SDK & Extension Architecture, defining how developers and approved external systems interact with OneDraft through supported clients, SDKs, plugins, APIs and extensibility mechanisms.

Status: ARCHITECTURALLY DEFINED

Next Document: Developer Platform, SDK & Extension Architecture Specification

Overall Architecture: OneDraft End-to-End System Integration